

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

SISTEMA WEB DE MONITOREO Y GESTIÓN DE LA INFRAESTRUCTURA DE REDES - ESPOCH

DOCUMENTO DE ESTÁNDAR Y METÁFORA DE MANEJO DE GIT

Flujo de Trabajo Colaborativo, Asignación de Roles por Desarrollador y Convenciones

Integrantes del Equipo:

- William Chisag
- Licet Ibarra
- Karen Maje
- Alejandro Rodríguez

Asignaturas y Docentes Responsables:

- Infraestructura TI: Ing. Tatiana Landivar
- Tecnología Web: Ing. Pedro Aguilar
- Conmutación y Enrutamiento: Ing. Juan Pazmiño
- Big Data: Ing. Geovanny Silva
- Diseño y Multimedia: Ing. Ramiro Bonilla

Período Académico: Quinto Semestre (PAO 5)

Documento Oficial de Ingeniería de Software (Versión 1.0)

Orellana - Ecuador

Octubre de 2026

Índice general

1. Metáfora del Control de Versiones: El Patio Ferroviario **4**

1.1. Concepto de la Metáfora: El Patio Ferroviario Central 4

2. Asignación de Roles y Responsabilidades por Desarrollador **6**

2.1. Responsabilidades Específicas de Cada Integrante 6

2.1.1. William Chisag (Líder Técnico & Backend Lead) 6

2.1.2. Karen Maje (Frontend & UI/UX Lead) 7

2.1.3. Licet Ibarra (Big Data, Base de Datos & QA Lead) 7

2.1.4. Alejandro Rodríguez (DevOps, Redes & Infraestructura Lead) . . . 7

3. Estrategia de Ramas y Ciclo de Vida del Software **8**

3.1. Estructura Jerárquica de Ramas 8

4. Estándar de Commits y Trazabilidad (Conventional Commits) **10**

4.1. Estructura Obligatoria del Mensaje 10

4.2. Tipos de Commit Permitidos 10

4.3. Reglas de Redacción de Commits 10

5. Flujo de Trabajo Paso a Paso y Comandos Prácticos **12**

5.1. Paso 1: Configuración Inicial de la Identidad del Desarrollador 12

5.2. Paso 2: Inicio de una Nueva Característica (Branching) 12

5.3. Paso 3: Desarrollo Local y Commits Atómicos 12

5.4. Paso 4: Sincronización Continua contra Develop 13

5.5. Paso 5: Publicación y Creación del Pull Request (PR) 13

5.6. Paso 6: Plantilla Obligatoria de Pull Request (Code Review) 13

5.7. Paso 7: Integración y Limpieza Local 14

6. Seguridad del Repositorio y Gestión de Archivos Ignorados **15**

6.1. Regla de Oro: Cero Secretos en el Repositorio 15

6.2. Archivo `.gitignore` Obligatorio 15

6.3. Plantilla de Variables de Entorno (`.env.example`) 16

7. Conclusiones **17**

Índice de tablas

2.1. Matriz de asignación de responsabilidades y ramas por usuario 6

4.1. Tipos estándar de commit y su aplicabilidad 11

Capítulo 1

Metáfora del Control de Versiones: El Patio Ferroviario

1.1 Concepto de la Metáfora: El Patio Ferroviario Central

Para organizar el desarrollo concurrente de cuatro ingenieros de software sin provocar colisiones de código, pérdidas de trabajo ni ramas rotas, se adopta la metáfora operativa de:

"La Red Ferroviaria y el Taller Central de Ensamblaje"

Bajo esta perspectiva:

- **La Vía Principal de Pasajeros (`main`):** Representa la línea férrea de alta velocidad en servicio comercial activo. Por aquí solo transitan trenes inspeccionados, certificados y listos para la demostración a los docentes y usuarios finales. Ningún mecánico puede realizar soldaduras ni pruebas directamente sobre la vía principal.
- **El Patio Central de Maniobras (`develop`):** Es la estación de pruebas integradas donde convergen los vagones terminados. Todo nuevo módulo se engancha primero aquí para verificar que los frenos, la energía y las señales electrónicas se sincronicen armónicamente entre todos los componentes.
- **Los Talleres de Construcción Individual (`feature/*`):** Son vías laterales cerradas donde cada maquinista ensambla una pieza específica (un motor, un panel de instrumentos o un sistema de radar). En su taller, el desarrollador puede desarmar y volver a armar su pieza con absoluta libertad sin descarrilar a los demás trenes.
- **La Inspección Técnica Obligatoria (*Pull Request & Code Review*):** Antes de que un vagón nuevo salga de su taller lateral hacia el patio central de maniobras, debe ser revisado por un jefe de taller (otro compañero del equipo) que comprueba que la soldadura cumpla los estándares de calidad y no existan fallas estructurales.

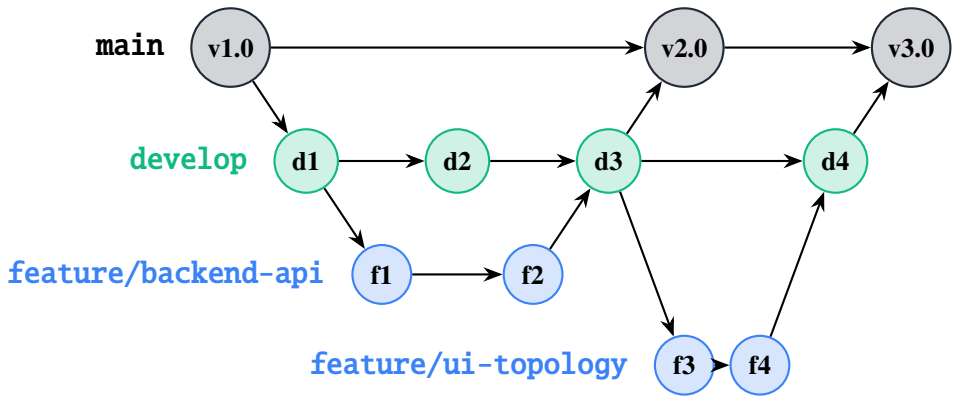


Figura 1.1: Topología del modelo de ramas Git adaptado para el equipo

Capítulo 2

Asignación de Roles y Responsabilidades por Desarrollador

Para maximizar la eficiencia y evitar solapamientos de código, se definen áreas de propiedad exclusiva (*Code Ownership*) según las competencias de cada integrante y los módulos del proyecto:

Tabla 2.1: Matriz de asignación de responsabilidades y ramas por usuario

Desarrollador	Rol Principal	Módulos Asignados	Prefijos de Rama Autorizados
William Chisag	Líder Técnico & Backend Lead	RF-01, RF-02, RF-05, RF-07, RF-10	`feature/auth-*`, `feature/daemon-ping`, `feature/api-config`
Karen Maje	Frontend & UI/UX Lead	RF-08, RF-09, RF-12, RNF-02, RNF-11	`feature/ui-dashboard`, `feature/ui-topology`, `feature/ui-inventory`
Licet Ibarra	Big Data, Base de Datos & QA Lead	RF-03, RF-04, RF-06, RF-11, RNF-08	`feature/db-models`, `feature/db-ttl-downsampling`, `feature/db-audit`
Alejandro Rodríguez	DevOps, Redes & Infraestructura Lead	RNF-01, RNF-03, RNF-07, RNF-14, RNF-15	`feature/devops-debian`, `feature/daemon-arpscan`, `feature/nginx-wireguard`

2.1 Responsabilidades Específicas de Cada Integrante

2.1.1 William Chisag (Líder Técnico & Backend Lead)

- **Gobernanza del Repositorio:** Administrador principal del repositorio en GitHub/-GitLab. Es responsable de proteger las ramas `main` y `develop`, asegurando que ninguna persona pueda realizar un `git push -force` que reescriba el historial compartido.
- **Capa de Negocio y Seguridad:** Desarrolla los endpoints de la API de Express, la autenticación dual con JWT (Access y Refresh tokens), el control de acceso basado en roles (RBAC) y la orquestación del daemon de disponibilidad (RF-01, RF-02, RF-05).
- **Aprobación Final de Merges:** Revisa y valida la arquitectura de los Pull Requests hacia `develop` antes de su integración definitiva.

2.1.2 Karen Maje (Frontend & UI/UX Lead)

- **Arquitectura de la Interfaz Web:** Estructura el proyecto React con Vite, configurando Tailwind CSS y el sistema de componentes modulares reutilizables.
- **Topología y Dashboard de Monitoreo:** Implementa el lienzo interactivo con React Flow para visualizar los enlaces físicos confirmados y las inferencias de red (RF-08), el centro de notificaciones in-app y el catálogo de filtros avanzados del inventario (RF-09, RF-12).
- **Verificación de Accesibilidad (RNF-11):** Garantiza que la interfaz cumpla con el estándar WCAG 2.1 AA, verificando contrastes de color, etiquetas de estado legibles para daltónicos e iconografía inequívoca.

2.1.3 Licet Ibarra (Big Data, Base de Datos & QA Lead)

- **Arquitectura de Persistencia NoSQL:** Diseña los esquemas y modelos de Mongoose en MongoDB, asegurando la consistencia entre dispositivos, eventos y alertas (RF-03, RF-06).
- **Retención y Analítica de Disponibilidad (Big Data):** Configura los índices TTL automáticos (30 días para comprobaciones brutas, 90 días para eventos) y programa los pipelines de agregación para el resumen diario (*downsampling*) de disponibilidad histórica (RF-11, Sección 8.4).
- **Control de Calidad (QA):** Diseña y ejecuta los planes de prueba de aceptación funcional, validando que los criterios de aceptación de cada historia de usuario se cumplan a cabalidad antes del cierre de cada sprint.

2.1.4 Alejandro Rodríguez (DevOps, Redes & Infraestructura Lead)

- **Infraestructura y Automatización del Servidor:** Administra la máquina Debian 12 en el laboratorio, configurando Nginx como proxy inverso, la segregación de puertos locales y la gestión de procesos con Systemd (`network-monitoring.service`).
- **Integración de Herramientas de Red de Bajo Nivel:** Desarrolla los módulos de ejecución nativa de `arp-scan` y `nmap` en Debian, gestionando permisos seguros de superusuario (*capabilities* de Linux sin exponer `sudo` generalizado) para el descubrimiento periódico de la subred (RF-04).
- **Acceso Remoto Criptográfico:** Configura el túnel de acceso seguro mediante WireGuard VPN para pruebas docentes externas seguras (Sección 8.8).

Capítulo 3

Estrategia de Ramas y Ciclo de Vida del Software

3.1 Estructura Jerárquica de Ramas

El equipo utilizará una variación pragmática y disciplinada de **GitFlow**:

1. **Rama `main` (Producción / Entregable Oficial):**
   - Contiene únicamente código estable, compilable y probado que corresponde a los hitos de entrega académica para los docentes.
   - Todo commit en `main` debe estar etiquetado con un tag semántico formal (ejemplo: `v1.0.0`, `v2.0.0`, `v3.0.0`).
   - **Regla estricta:** Nadie hace push directo a `main`. Los cambios únicamente ingresan mediante Pull Request desde `develop` tras la aprobación del equipo.
2. **Rama `develop` (Integración Continua de Desarrollo):**
   - Es el tronco de trabajo activo. Refleja el estado del sprint en curso con todas las características validadas hasta la fecha.
   - Sirve de punto de partida para cualquier nueva característica y destino de todos los Pull Requests de desarrollo.
3. **Ramas de Característica (`feature/<modulo>-<nombre-descriptivo>`):**
   - Se crean siempre a partir de `develop`.
   - Tienen una vida útil corta (idealmente entre 1 y 4 días de trabajo).
   - No deben contener trabajo de múltiples requerimientos desvinculados.
   - **Ejemplos válidos:**
     - `feature/auth-jwt-refresh` (William Chisag)
     - `feature/ui-network-topology` (Karen Maje)
     - `feature/db-ttl-aggregation` (Licet Ibarra)
     - `feature/daemon-arp-discovery` (Alejandro Rodríguez)
4. **Ramas de Corrección de Defectos (`bugfix/<descripcion>`):**
   - Se desprenden de `develop` para solucionar fallos identificados durante las pruebas internas de integración.
   - Ejemplo: `bugfix/fix-ping-timeout-calculation`.
5. **Ramas de Parche Crítico (`hotfix/<version>`):**

- Se originan directamente desde `main` únicamente ante una falla catastrófica descubierta en plena evaluación o demostración en vivo.
- Una vez corregido el problema, se integran tanto a `main` como a `develop`.
- Ejemplo: `hotfix/v1.0.1-crash-on-empty-mac`.

Capítulo 4

Estándar de Commits y Trazabilidad (Conventional Commits)

Para garantizar la trazabilidad exigida por la norma ISO/IEC/IEEE 29148 y facilitar la generación automatizada de bitácoras de cambio (*Changelog*), todos los commits deben adherirse estrictamente a la especificación **Conventional Commits v1.0.0**.

4.1 Estructura Obligatoria del Mensaje

```
<tipo>(<alcance>): <descripcion concisa en imperativo> [#<
   id_requerimiento>]

[Cuerpo opcional que explica el porqué del cambio y alternativas
   consideradas]

[Pie opcional: Referencias a incidencias o avisos de cambios
   incompatibles]
```

4.2 Tipos de Commit Permitidos

4.3 Reglas de Redacción de Commits

- **Modo imperativo:** Escribir en tiempo presente (ej. “agregar”, “corregir”, “eliminar”), nunca en pasado (“agregado”, “corregí”).
- **Primera letra en minúscula:** Después de los dos puntos del prefijo, la descripción inicia en minúscula.
- **Sin punto final:** La línea principal no debe terminar con punto.
- **Trazabilidad vinculante:** Siempre que el commit satisfaga o modifique un requerimiento del backlog, se debe incluir su identificador entre corchetes (ej. `[#RF-05.02]`).

Tabla 4.1: Tipos estándar de commit y su aplicabilidad

Prefijo	Significado Semántico	Ejemplo Aplicado al Proyecto
`feat`	Incorporación de una nueva funcionalidad o requerimiento	`feat(auth): agregar rotacion dual con refresh tokens [#RF-01]`
`fix`	Corrección de un defecto o error de software	`fix(daemon): resolver falso positivo por timeout en WiFi [#RF-05]`
`perf`	Optimización de rendimiento o consumo de recursos	`perf(db): agregar indice TTL para depurar comprobaciones [#RNF-15]`
`refactor`	Reestructuración de código sin alterar su comportamiento	`refactor(api): modularizar controladores de inventario [#RNF-12]`
`style`	Ajustes de formato, espaciado o interfaz sin cambio lógico	`style(ui): estandarizar bordes y badges de estado [#RNF-11]`
`docs`	Modificaciones en documentación técnica o comentarios	`docs(api): documentar endpoints con OpenAPI/Swagger`
`test`	Incorporación o corrección de pruebas automatizadas	`test(auth): anadir pruebas de bloqueo tras 3 fallos [#RF-01.07]`
`chore`	Mantenimiento de dependencias, scripts de build o CI/CD	`chore(deps): actualizar Tailwind CSS y dependencias Vite`

Capítulo 5

Flujo de Trabajo Paso a Paso y Comandos Prácticos

A continuación se detalla la guía operativa que cada desarrollador ejecutará en su terminal de Linux para trabajar colaborativamente de manera segura:

5.1 Paso 1: Configuración Inicial de la Identidad del Desarrollador

Al clonar el repositorio en su computadora personal o en el laboratorio, cada integrante debe verificar que su nombre y correo institucional estén registrados:

```
# Ejemplo para William Chisag:
git config --global user.name "William␣Chisag"
git config --global user.email "william.chisag@espoch.edu.ec"

# Configuracion recomendada de ayuda de merge y estilo:
git config --global pull.rebase true
git config --global core.autocrlf input
```

5.2 Paso 2: Inicio de una Nueva Característica (Branching)

Antes de crear una rama, siempre se debe sincronizar `develop` con el servidor remoto:

```
git checkout develop
git pull origin develop
git checkout -b feature/daemon-icmp-ping
```

5.3 Paso 3: Desarrollo Local y Commits Atómicos

El desarrollador programa su solución y realiza commits pequeños y autocontenidos:

```
git status
git add src/services/pingService.js
git commit -m "feat(daemon):␣implementar␣sondeo␣ICMP␣con␣timeout␣de
   ␣3s␣[#RF-05.03]"
```

5.4 Paso 4: Sincronización Continua contra Develop

Para evitar acumulaciones masivas de conflictos, antes de publicar la rama se debe rebasar contra los últimos cambios de los compañeros en `develop`:

```
git fetch origin
git rebase origin/develop
```

Si existen conflictos: Git pausará el proceso. El desarrollador edita los archivos en conflicto, guarda las versiones correctas y continúa:

```
git add src/services/pingService.js
git rebase --continue
```

5.5 Paso 5: Publicación y Creación del Pull Request (PR)

```
git push -u origin feature/daemon-icmp-ping
```

El desarrollador ingresa a GitHub/GitLab y abre un Pull Request dirigido estrictamente hacia la rama **`develop`**.

5.6 Paso 6: Plantilla Obligatoria de Pull Request (Code Review)

Todo PR debe incluir en su descripción la siguiente información estandarizada:

```
### Descripcion del Cambio:
Se implemento el servicio de comprobacion ICMP en segundo plano con
limite de concurrencia de 15 hilos mediante p-limit.

### Requerimientos Asociados:
- RF-05.01 (Servicio independiente)
- RF-05.03 (Timeout de 3 segundos)
- RNF-15 (Uso eficiente de recursos)

### Verificacion y Pruebas Realizadas:
- [x] Prueba de ping exitosa sobre 192.168.1.1 (Router).
- [x] Prueba de caida simulada desconectando host (3 fallos -> No
   disponible).
- [x] Verificado que no existen fugas de sockets en Debian.

### Revisor Asignado:
@WilliamChisag / @AlejandroRodriguez
```

5.7 Paso 7: Integración y Limpieza Local

Una vez que el revisor aprueba el PR y se ejecuta el merge hacia `develop`, el autor elimina la rama local para mantener limpio su entorno:

```
git checkout develop
git pull origin develop
git branch -d feature/daemon-icmp-ping
```

Capítulo 6

Seguridad del Repositorio y Gestión de Archivos Ignorados

6.1 Regla de Oro: Cero Secretos en el Repositorio

Bajo las recomendaciones de ISO/IEC 27001, está terminantemente prohibido subir al control de versiones credenciales, contraseñas de bases de datos, claves maestras de JWT o certificados privados.

6.2 Archivo `.gitignore` Obligatorio

En la raíz del proyecto debe existir un archivo `.gitignore` con la siguiente configuración mínima obligatoria:

```
# Dependencias de paquetes
node_modules/
dist/
build/

# Archivos de entorno y secretos
.env
.env.local
.env.production
*.pem
*.key

# Logs del sistema operativo y herramientas
logs/
*.log
npm-debug.log*
yarn-debug.log*

# Archivos generados por el compilador LaTeX
*.aux
*.lot
*.toc
*.log
*.out
*.synctex.gz
```

```
*.fls
*.fdb_latexmk

# Archivos temporales de sistemas operativos y editores
.DS_Store
Thumbs.db
.vscode/
.idea/
```

6.3 Plantilla de Variables de Entorno (`.env.example`)

Para que cualquier integrante del equipo pueda configurar su entorno de desarrollo local sin comprometer la seguridad, se versiona un archivo público `.env.example` con valores ficticios:

```
# Configuracion del Servidor API
PORT=5000
NODE_ENV=development

# Base de Datos MongoDB
MONGO_URI=mongodb://127.0.0.1:27017/network_monitoring

# Seguridad Criptografica JWT
JWT_ACCESS_SECRET=
   clave_secreta_super_segura_de_desarrollo_access_token_123
JWT_REFRESH_SECRET=
   clave_secreta_super_segura_de_desarrollo_refresh_token_456
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=8h

# Parametros de Red y Monitoreo Predeterminados
DEFAULT_PING_INTERVAL_SEC=30
DEFAULT_DISCOVERY_INTERVAL_MIN=10
DEFAULT_PING_TIMEOUT_MS=3000
MAX_CONCURRENT_PINGS=15
```

Capítulo 7

Conclusiones

La disciplina en el control de versiones constituye el pilar que garantiza la integridad, la autoría y el progreso continuo del proyecto integrador. El cumplimiento riguroso de este estándar de Git permitirá al equipo de cuatro integrantes desarrollar de forma paralela sin bloqueos ni retrasos, entregando a la ESPOCH una solución de calidad profesional.